

Backtracking

In the search for fundamental principles of algorithm design, backtracking represents one of the most general techniques. Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using the backtracking formulation. In many applications of the backtrack method, the desired solution is expressible as an n -tuple $(x_1, \dots, x_n)$, where the x_i are chosen from some finite set S_i . Often the problem to be solved calls for finding one vector that maximizes (or minimizes or satisfies) a criterion function $P(x_1, \dots, x_n)$. Sometimes it seeks all vectors that satisfy P . For example, sorting the array of integers in $a[1:n]$ is a problem whose solution is expressible by an n -tuple, where x_i is the index in a of the i th smallest element. The criterion function P is the inequality $a[x_i] \leq a[x_{i+1}]$ for $1 \leq i \leq n$. The set S_i is finite and includes the integers 1 through n . Though sorting is not usually one of the problems solved by backtracking, it is one example of a familiar problem whose solution can be formulated as an n -tuple.

The brute force approach would be to form all these n -tuples, evaluate each one with P , and save those which yield the optimum. The backtrack algorithm has as its virtue the ability to yield the same answer with far fewer than all trials. Its basic idea is to build up the solution vector one component at a time and to use modified criterion functions $P_i(x_1, \dots, x_i)$ (sometimes called bounding functions) to test whether the vector being formed has any chance of success. The major advantage of this method is this: if it is realized that the partial vector $(x_1, x_2, \dots, x_i)$ can in no way lead to an optimal solution further test vectors can be ignored entirely.

Sum of subsets

Problem: Given n positive integers $w_1, \dots, w_n$ and a positive integer S . Find all subsets of $w_1, \dots, w_n$ that sum to S .

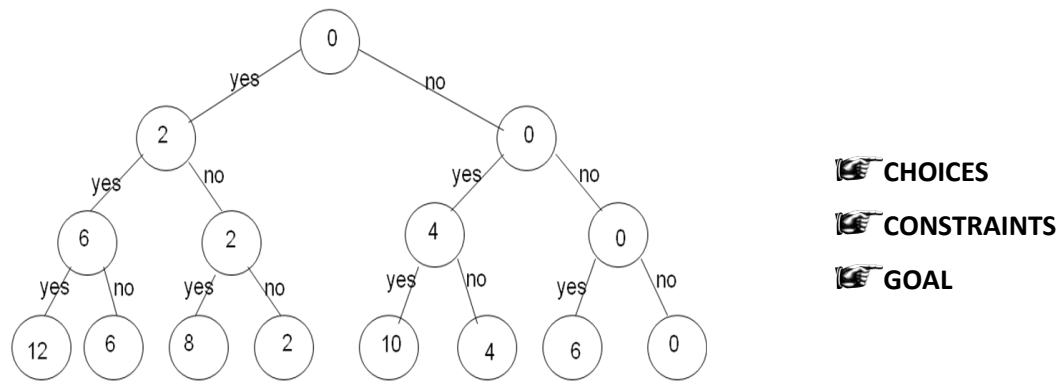
Example:

$n=3$, $S=6$, and $w_1=2$, $w_2=4$, $w_3=6$

Solutions:

$\{2,4\}$ and $\{6\}$

We will assume a binary state space tree. The nodes at depth 1 are for including (yes, no) item 1, the nodes at depth 2 are for item 2, etc. The left branch includes w_i , and the right branch excludes w_i . The nodes contain the sum of the weights included so far



The sum of the included integers is stored at the node.

Problems can be solved using depth first search of the (implicit) state space tree. Each node will save its depth and its (possibly partial) current solution. DFS can check whether node v is a leaf.

- If it is a leaf then check if the current solution satisfies the constraints
- Code can be added to find the optimal solution

Such a DFS algorithm will be very slow. It does not check for every solution state (node) whether a solution has been reached, or whether a partial solution can lead to a feasible solution. We call a node non-promising if it cannot lead to a feasible (or optimal) solution, otherwise it is promising. Main idea: Backtracking consists of doing a DFS of the state space tree, checking whether each node is promising and if the node is nonpromising backtracking to the node's parent

When is a node “promising”?

Consider a node at depth i

weightSoFar = weight of node, i.e., sum of numbers included in partial solution node represents

totalPossibleLeft = weight of the remaining items $i+1$ to n (for a node at depth i)

A node at depth i is non-promising

if $(\text{weightSoFar} + \text{totalPossibleLeft} < S)$ // insufficient sum

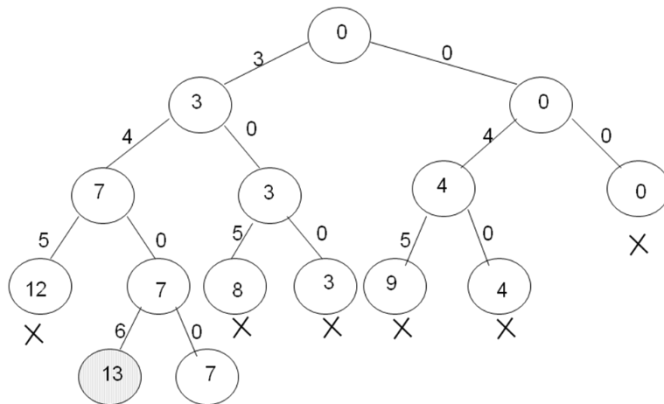
or $(\text{weightSoFar} + w_{i+1} > S)$ // excessive sum

To be able to use this “promising function” the w_i must be sorted in non-decreasing order

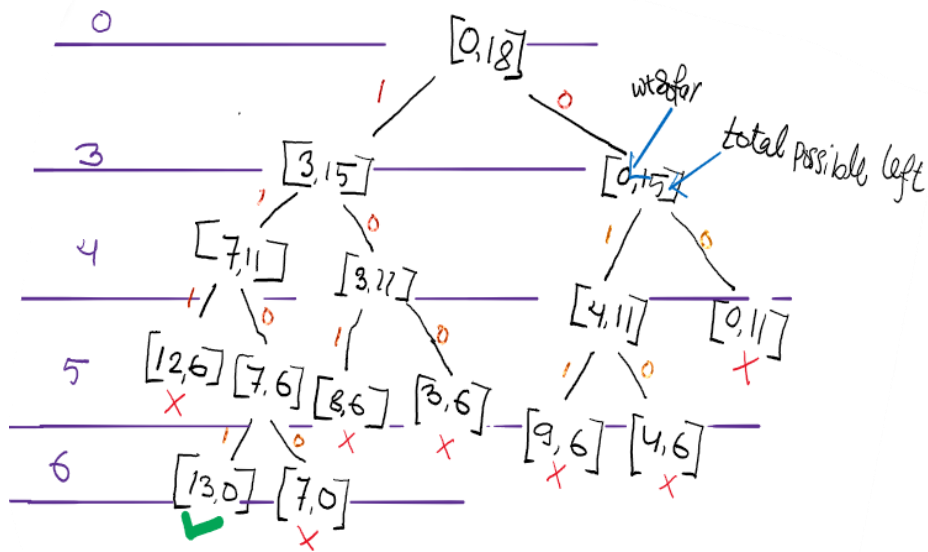
The state space tree consisting of expanded nodes only is called the pruned state space tree. In the example given below, there are only 15 nodes in the pruned state space tree. The full state space tree has 31 nodes

A Pruned State Space Tree (find all solutions)

$w_1 = 3, w_2 = 4, w_3 = 5, w_4 = 6; S = 13$



Sum of subsets problem



Sum of sub-set problem state-space tree.

```

sumOfSubsets ( i, weightSoFar, totalPossibleLeft )
{
    if (promising ( i )) //may lead to solution
    {include [i+1]="yes"
      }
    include [ i + 1 ] = "no"           //try excluding
    sumOfSubsets ( i + 1, weightSoFar , totalPossibleLeft - w[i + 1] )
}

boolean promising ( i )
{
    return (weightSoFar + totalPossibleLeft ≥ S) &&( weightSoFar == S || weightSoFar +
w[i + 1] ≤ S )
}

```

0/1 knapsack Problem

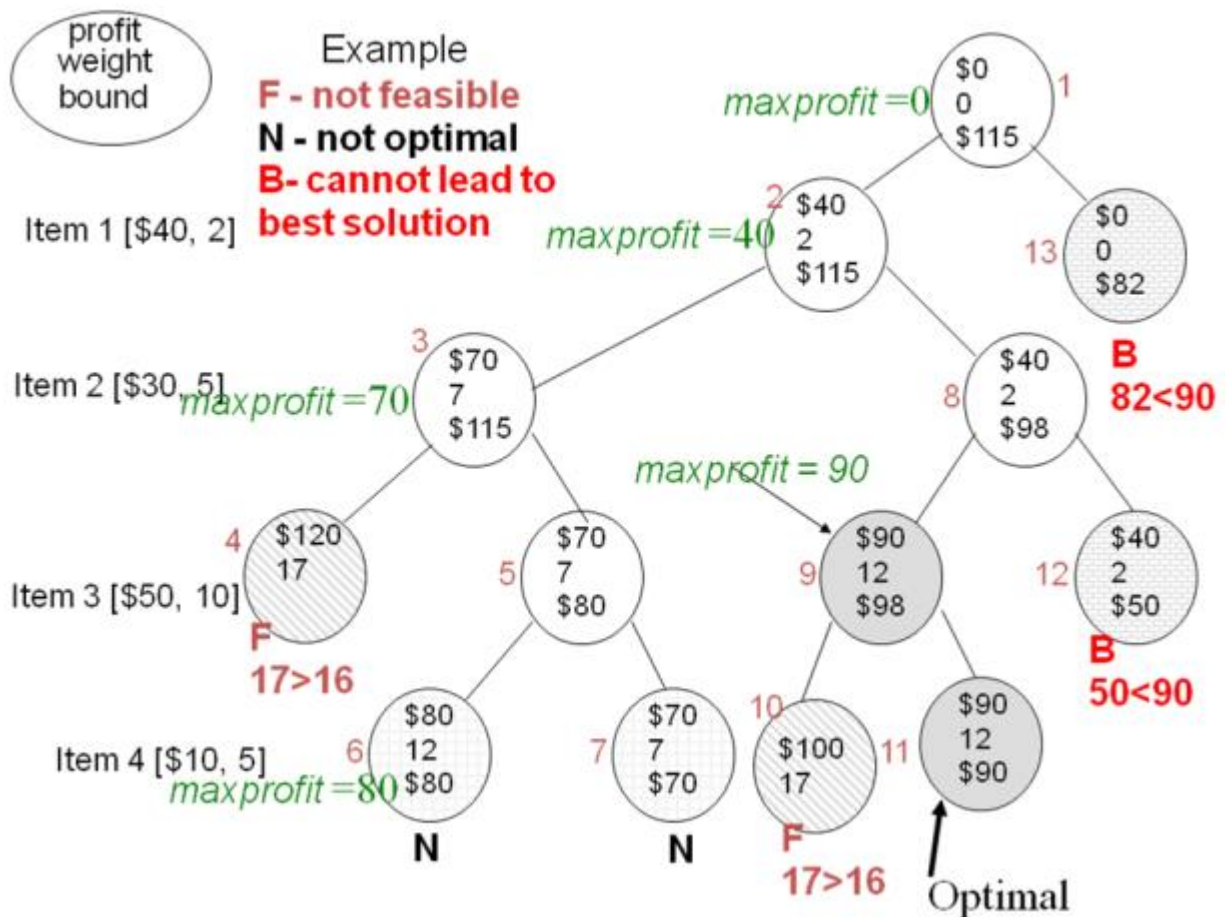
Each node v in the state space will include 3 values:

- profit (v) = sum of profits of all items included in the knapsack (on a path from root to v)
- weight (v) = the sum of the weights of all items included in the knapsack (on a path from root to v)
- upperBound(v) = upperBound(v) is greater or equal to the maximum benefit that can be found by expanding the whole subtree of the state space tree with root v .

The nodes are numbered in the order of expansion

Suppose $n = 4$, $W = 16$, and we have the following:

i	p_i	w_i	p_i / w_i
1	\$40	2	\$20
2	\$30	5	\$6
3	\$50	10	\$5
4	\$10	5	\$2



Calculation for node 1:

maxprofit = \$0 (n = 4, C = 16)

profit = \$0

weight = 0

$$\begin{aligned} \text{bound} &= \text{profit} + p_1 + p_2 + (C - 7) * p_3 / w_3 \\ &= \$0 + \$40 + \$30 + (16 - 7) * \$50 / 10 = \$115 \end{aligned}$$

Node 1 is promising because its weight = 0 < C = 16
 and its bound \$115 > 0 (the value of maxprofit.)

Calculation for Node 2:

Item 1 with profit \$40 and weight 2 is included

$$\text{maxprofit} = \$40$$

$$\text{profit} = \$40$$

$$\text{weight} = 2$$

$$\begin{aligned}\text{bound} &= \text{profit} + p_2 + (\mathbf{C} - 7) \times p_3 / w_3 \\ &= \$40 + \$30 + (16 - 7) \times \$50/10 = \$115\end{aligned}$$

Node 2 is promising because its weight = 2 < **C** = 16
and its bound \$115 > \$40 (the value of maxprofit.)

Calculation for Node 13:

Item 1 with profit \$40 and weight 2 is not included

At this point maxprofit=\$90 and is not changed

$$\text{profit} = \$0$$

$$\text{weight} = 0$$

$$\begin{aligned}\text{bound} &= \text{profit} + p_2 + p_3 + (\mathbf{C} - 15) \times p_4 / w_4 \\ &= \$0 + \$30 + \$50 + (16 - 15) \times \$10/5 = \$82\end{aligned}$$

Node 13 is nonpromising because its bound \$82 < \$90 (the value of maxprofit).

Calculation for Node 5:

Item 3 with profit \$50 and weight 10 is not included

$$\text{maxprofit} = \$70$$

$$\text{profit} = \$70$$

weight = 7 // room for next item whole item can be taken

$$\begin{aligned}\text{bound} &= \text{profit} + p_4 \\ &= \$70 + \$10 \\ &= \$80\end{aligned}$$

knapsack(i, profit, weight)

```

{
    if ( weight <= C && profit > maxprofit)
    {
        // save better solution
        maxprofit=profit //save new profit
        numbest= i; bestset = include//save solution
    }
    if (promising(i))
    {
        include [i + 1] = “ yes”
        knapsack(i+1, profit+p[i+1], weight+ w[i+1])
        include[i+1] = “no”
        knapsack(i+1, profit,weight)
    }
}

promising(i)
{
    //Cannot get a solution by expanding node
    if weight >= C return false
    //Compute upper bound
    bound = Calculate_Bound(i+1, weight, profit, w, p, C, n)
    return (bound>maxprofit)
}

Calculate_Bound(i, weight, profit, w, p, C, n)
{
    bound = profit
    for j=i to n
        x[j]=0 //initialize variables to 0
    while (weight<C)&& (i<=n)    //not “full”and more items
    {

```



```

    if weight+w[i]<=C           //room for next item
    {
        x[i]=1                 //item i is added to knapsack
        weight=weight+w[i]; bound = bound +p[i]
    }
    else
    {
        x[i]=(C-weight)/w[i] //fraction of i added to knapsack
        weight=C; bound = bound + p[i]*x[i]
        i=i+1                  // next item
    }
}

return bound
}

```

Time Complexity : $T(n) = ??$

N-Queen Problem

A chess board has $N \times N$ fields. Is it possible to place n queens on this board, so that no two queens can attack each other? A queen can attack horizontally, vertically, and on both diagonals, so it is pretty hard to place several queens on one board so that they don't attack each other.

Example: The 4-queens problem

Basic idea of solution:

- Start with one queen in the first column, first row.
- Start with another queen in the second row, first column.
- Go down left with the second queen until you reach a permissible situation.
- Advance to the next row, first column, and do the same thing.
- If you cannot find a permissible situation in one column and reach the bottom of it, then you have to go back to the previous column and move one position down there.
(This is the backtracking step.)

If you reach a permissible situation in the last column of the board, then the problem is solved. If you have to backtrack BEFORE the first column, then the problem is not solvable.

Obviously, in any solution to the n -Queens problem, there is exactly one queen in each row. So we will represent our possible solutions using an array $Q[1 \dots n]$, where $Q[i]$ indicates which square in row i that contains a queen, or 0 if no queen has yet been placed in row i . To find a solution, we put queens on the board row by row, starting at the top. A partial solution is an array $Q[1 \dots n]$ whose first $r-1$ entries are positive and whose last $n-r+1$ entries are all zeros, for some integer r .

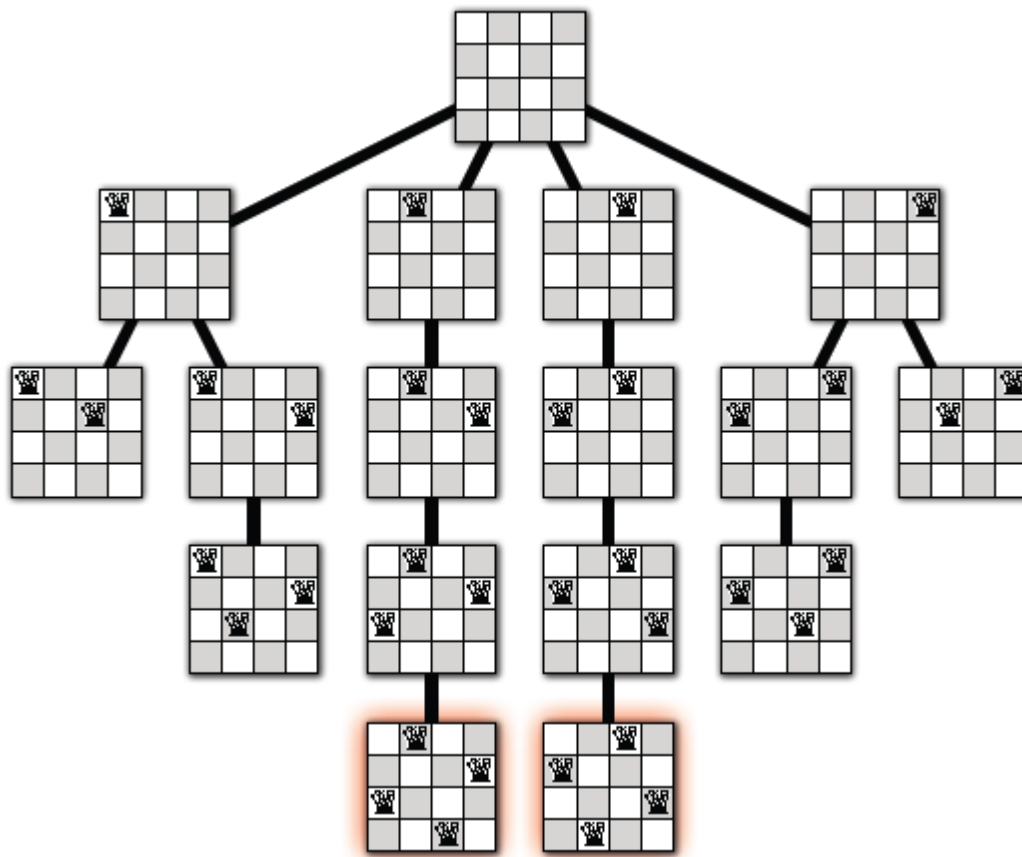


Fig: Pruned State Space tree.

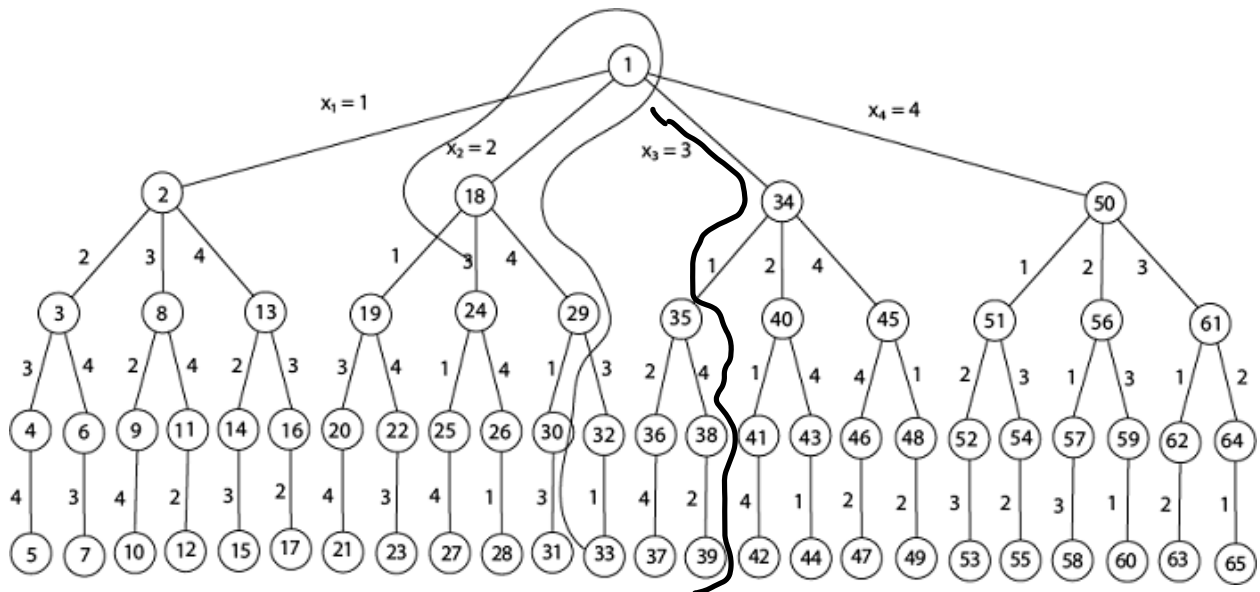


Fig : Complete State Space Tree without backtracking solution vectors are [2,4,1,3] & [3,1,4,2]

The following recursive algorithm recursively enumerates all complete n -queens solutions that are consistent with a given partial solution. The input parameter r is the first empty row. Thus, to compute all n -queens solutions with no restrictions, we would call `RECURSIVENQUEENS(Q[1 .. n], 1)`.

`NQueens(Q[1 .. n], r)`

{

 if $r = n+1$

 print Q

 else

 for $j = 1$ to n

 promising=true

 for $i = 1$ to $r-1$

 if $(Q[i] = j) \text{ or } (Q[i] = j + r - i) \text{ or } (Q[i] = j - r + i)$

 promising=false

 if promising=true

$Q[r] = j$

`NQueens(Q[1 .. n], r + 1)`

}

Complexity : $T(n) = ??$